NES AI FINANCE & INSURANCE INTELLIGENCE PLATFORM

Cline Project Instructions

Document ID: NES-CLINE-006 | **Version:** 1.0

Build approach: Cline-assisted development • Local-first Windows application

Scope: Personal research, paper trading and insurance document assistance

Repository conventions, implementation discipline and boundaries

1. Purpose

This document is intended to be placed in the repository as the project-specific instruction source for Cline. It is guidance for implementation, not a prompt to execute any particular task.

2. Product boundaries

- Build a personal-use, local-first finance and insurance intelligence application.
- Initial trading scope is historical research and paper trading only.
- Never implement live broker order routing or insurance purchases in the initial release.
- Never label demo data as live or imply simulated results are real.
- Do not send personal documents to external services without explicit approval and clear disclosure.

3. Engineering rules

- Inspect the existing repository before changing architecture or replacing files.
- Prefer small, focused changes that can be run and tested.
- Use TypeScript types and runtime validation at system boundaries.

- Reuse shared UI components and central design tokens.
- Keep domain logic outside presentation components.
- Avoid giant single-file components; separate by domain and responsibility.
- Do not add dependencies without a clear need; explain large additions.
- Do not overwrite working configuration without inspecting it first.
- Do not invent API endpoints, provider capabilities or credentials.

4. UI and motion rules

- Follow the hybrid theme: light overview/insurance and optional dark trading.
- Use Inter, forest green, restrained orange, readable spacing and consistent cards.
- Implement all 12 animations as purposeful, replayable demonstrations.
- Use GSAP for complex timelines and CSS for basic transitions.
- Clean up GSAP timelines and ScrollTriggers on component unmount.
- Respect reduced-motion settings and provide static fallbacks.
- Three.js must be lazy-loaded and optional; core UI works without WebGL.

5. Data and safety rules

- Use clearly labelled synthetic/demo data until a real provider is selected and verified.
- Never hardcode a claim that data is live.
- Never store secrets in frontend code, source control or normal logs.
- Do not mark extracted policy fields verified without a review step.
- Keep paper trading visibly separate from real trading.
- Financial calculations must be in testable domain modules with documented assumptions.

6. Required workflow for coding tasks

- Read the relevant project docs and inspect related files first.
- Identify the smallest scope and list likely files to change.
- Implement the change while preserving existing working behaviour.
- Run available lint, type-check, tests and build commands.
- Fix regressions introduced by the change.
- Summarise files changed, tests run, known limitations and any manual test needed.
- If blocked by missing credentials/provider details, create an explicit adapter interface and document the blocker instead of fabricating connectivity.

7. Definition of done

- The requested behaviour is implemented, not merely described.
- No broken navigation or inert primary controls remain in the changed flow.
- Relevant tests/build pass, or failures are reported precisely.
- Documentation and changelog are updated when behaviour or setup changes.
- The change preserves local-first and personal-use boundaries.

These are standing repository instructions. The actual implementation task should be supplied separately when the project owner is ready.